

Multiparty Lowest Unique Positive Integer (LUPI) in Ren

Li Xuan Tan

August 21, 2026

1 Implementation

N players using the Rust client DKG-ing against a combined accumulation and computation server over office WiFi. Blue's LUPI algorithm follows:

- Encode each player's two input integers in a "two-hot" vector, where slot i is 1 iff the player chose i as one of their integers. The maximum allowed integer S is both a game design/strategy choice, as well as a technical consideration (large S will increase the number of rotation keys and depth).
- Form a binary tree with N encoded inputs, then apply the "saturating" polynomial that takes $[0, 1, 2, 3, 4] \rightarrow [0, 1, 2, 2, 2]$ to fold the two inputs at each node. This basically keeps 0 = unpicked, 1 = uniquely picked, 2 = multiply picked, at a cost of $3 \log_2(N)$ multiplicative depth.
- Apply the polynomial taking $[0, 1, 2] \rightarrow [0, 1, 0]$ so that the only remaining 1s in the fully-folded ciphertext are in the slots representing unique choices.
- To find the lowest unique choice, we take the prefix product $ct[i] * \prod_{j < i} (1 - ct[j])$ so that an index is 1 iff it is 1 and no earlier index is 1. This can be done by rotate-and-multiply with 2^j step size in $\log_2(S)$ depth.
- Now to obtain the winner ID, we compute $\sum_{n < N} input[n] * id[n]$ for all players and multiply this by the ciphertext from the previous step.
- Rotate-and-sum across all slots to get the winner ID in all slots (or not, if revealing the winning number is desired).

1.1 Trust models

Decided on assuming server is honest but curious, since we have to rely on the server to perform compute faithfully anyway (in the absence of ZKP for CKKS). Implications:

- No commit-reveal ceremony needed for joint randomness during DKG - the `ren-multiparty-examples` repo has this if needed
- No cross-verification of individual users' registrations - everyone trusts the server's roster
- Not wanting the server to see the final result means that decryption shares are encrypted with a (non-FHE) sealing key and shared between each pair of users; the server only serves as a relay and not an accumulator during decryption.
- Web client is trusted to serve exactly the compiled WASM, instead of a modified page that extracts players' secret shards. This is an additional attack surface over compiled and distributed binaries with checksums.

2 Multiparty learnings

2.1 Choosing parameters

High `log_precision` is needed to have enough bits for noise flooding, which means you won't have much depth (trying to get more depth is bloody expensive - bootstrapping will be needed).

If you use many parties, the num-squarings formula may break (there is a current hard-cap in `params.py` that Noah thinks we might be able to remove) and you'll have to raise exp-degree for boot params in order to get acceptable precision, further reducing available depth.

There are also some "dead zones" where ren won't build the prime chain for specific values of `lp`, possibly related to hardcoded boot-prime-bits values?

2.2 t-of-n vs n-of-n

Tradeoffs of dropout resistance (a *n-of-n* policy means any dropout ends the entire set of games as well as blocking the current game from decrypting) vs privacy (a *t-of-n* policy lowers the bound to collusion and means an unwilling player could have their input decrypted).

Is the noise flooding formula from eprint 2024/681 still valid under threshold decryption? Believed to be true but uncertain. Empirical calibration showed circuit noise to be very similar.

2.3 Noise flooding calibration

Eprint 2024/681 states that the noise-flooding stdev should be $\sigma = \sqrt{12D} \cdot 2^{\nu/2} \cdot noise_{ct}$ where D is the number of decryptions and ν is statistical security. To obtain the circuit noise, we currently do an empirical test where we decode unflooded circuit outputs from public dummy data and observe the noise (since noise tracking is borked?). This should ideally be done over a large number of

keysets and sample data covering worst-case scenarios (eg. largest ciphertext magnitude), and should be recalibrated for circuit changes.

Relatedly, choose your encodings carefully (the sum-of-IDs encoding with $N=25$ caused noise to grow significantly)

If threshold decryption is used, D should count the number of quorum formations (which are "observations" of one ciphertext) - players should keep their own record of how many times they've published flooded views of their secret shard

2.4 Secret key share serialization

Currently doesn't exist upstream in ren, so there is no way to keep a player's secret key share outside of process (ie. it dies with a Ctrl-C or tab closure). Re-derivation from stored seeds is currently the mitigation, but takes 1-2min.

2.5 High-precision vs regular bootstrapping

High-precision or meta-bootstrapping takes a long time and is the main component of an un-JITted multiparty circuit's runtime (≈ 50 s per meta-bts).

Either JIT the meta-bts or do some jank where some of your bootstraps in non-noise-critical parts (eg. the saturating function which you can bit-clean) are regular ones instead of meta-bts. This "hybrid" mode calibrated at basically identical noise levels to all-meta, but unproven mathematically. Not currently supported in upstream ren (all bts calls are meta-bts if meta params are set).

To improve UX, JITting was done on a server-generated set of dummy inputs (approx 90s) immediately after DKG, while players are doing their first inputs.

3 Client-side learnings

3.1 DKG bottlenecks

Primary bottleneck most of the time was a single slow client - no one can proceed to stage 2 while the slowest client is still doing stage 1. WASM doesn't seem to affect the relative speed.

An early concern was whether office WiFi would handle up to $N = 25$ clients doing simultaneous large uploads/downloads. It seemed to hold up fine, although a slowdown was observed from ≈ 45 MBps at the beginning to ≈ 16 MBps at the end. Source of this slowdown is unclear/being investigated; it doesn't fit the expected pattern from network congestion since there were no spikes when faster clients finished, but a server-side test ruled out slowdowns there.

Related to the above, if doing DKG over internet, make sure large uploads/downloads are chunked for better resilience, otherwise you risk having to start from scratch on gigabyte-scale uploads. This also serves as a good proxy to show users an upload progress bar (which may not be doable for keygen), which is generally good UX. Also use polling instead of call+await response which

times out easily on large computations (on uploads, this is another reason to use chunks).

3.2 Cross-platform support

Two paths we took: compiled binaries for each OS (can be annoying getting people to recompile each time - either set up one of the office mini PCs to compile for Windows or bug Blue/other Windows users) or browser-served WASM.

WASM however caps at 4GB RAM, which may not be enough (LUPI DKG used up to 3.4GB during keygen, and Andrii's elections overflowed). Tabbing away also leads many browsers/OSes to kill a high-memory-use tab, which then requires rederiving keysets - bad UX. Streaming encode is a possible solution to reduce RAM use since currently the full keyset has to be held in memory as it's being encoded, but would probably be an upstream change.

As usual, cross-platform crypto can differ between backends (eg. ChaCha20 returning different results on the same seeds on GPU vs CPU). `lupi preflight` pins an expected reference and checks that the local crypto works.

3.3 Communication costs

$O(N^2)$ during decode is required in order to keep things private from accumulator (each client passes its sealed share to each other client). These are relatively small shares of a few MB each, so network congestion is not a problem here, but merits discussion especially if scaling up to very large N .

As of now, the main communication cost remains DKG stage 1. The KG+/BTS+ keygen paper is being looked into to reduce communication cost by sending smaller "transmission keys" from which the evaluation keys are then derived.

Nice-to-have for aggregator server: incremental aggregation (currently not supported) as shares come in, to avoid having to hold $N=25$ round-1 DKG messages of size up to 1.8GB all in RAM simultaneously. Fine on apollo/artemis with 700+GB RAM, but not on a mini PC where we might have the aggregator if it's eventually separated from the evaluation server.

3.4 Using HTTPS

HTTPS-secured sites (or accessing localhost) are needed for certain WASM features like persistent OPFS or crypto. Some tailscale magic was needed here to serve a secure website without having to register our own certificate - Skylar has some more details on this, but it involves using `tailscale serve` to securely forward the server port onto a tailnet URL.